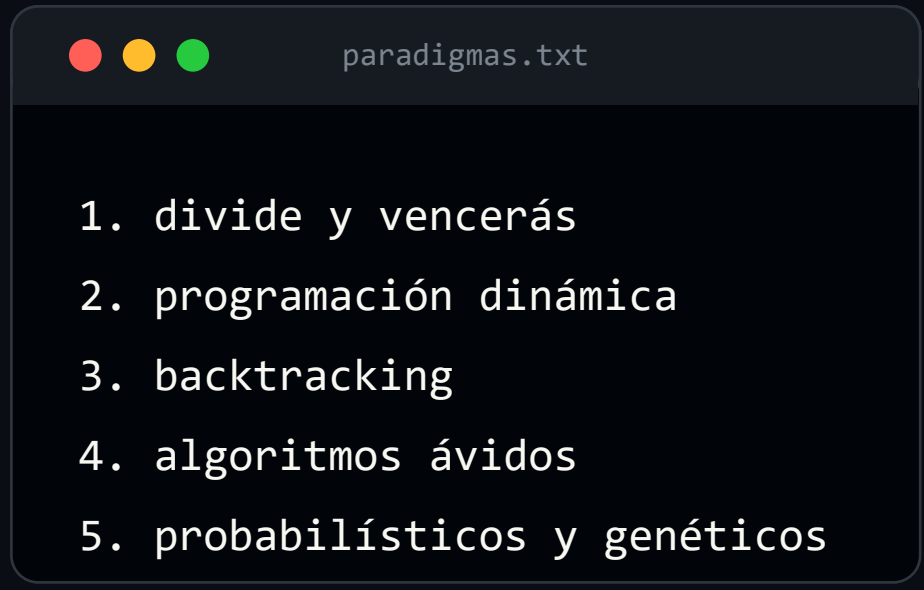
CE-2103 · Lección 05

Diseño de Algoritmos

Paradigmas para resolver nuevos problemas



Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Aplicar divide y vencerás con Merge Sort y búsqueda binaria
2. Evitar subproblemas repetidos con programación dinámica (mochila, monedas)
3. Resolver problemas con backtracking y algoritmos ávidos
4. Diferenciar Monte Carlo de Las Vegas y usar filtros de Bloom
5. Describir cómo funciona un algoritmo genético (ejemplo: la mochila)

// divide y vencerás

Divide y vencerás: tres fases

1

Dividir

Parte el problema en subproblemas más pequeños, hasta llegar al caso base.

2

Conquistar

Resuelve cada subproblema por separado; los pequeños, de forma directa.

3

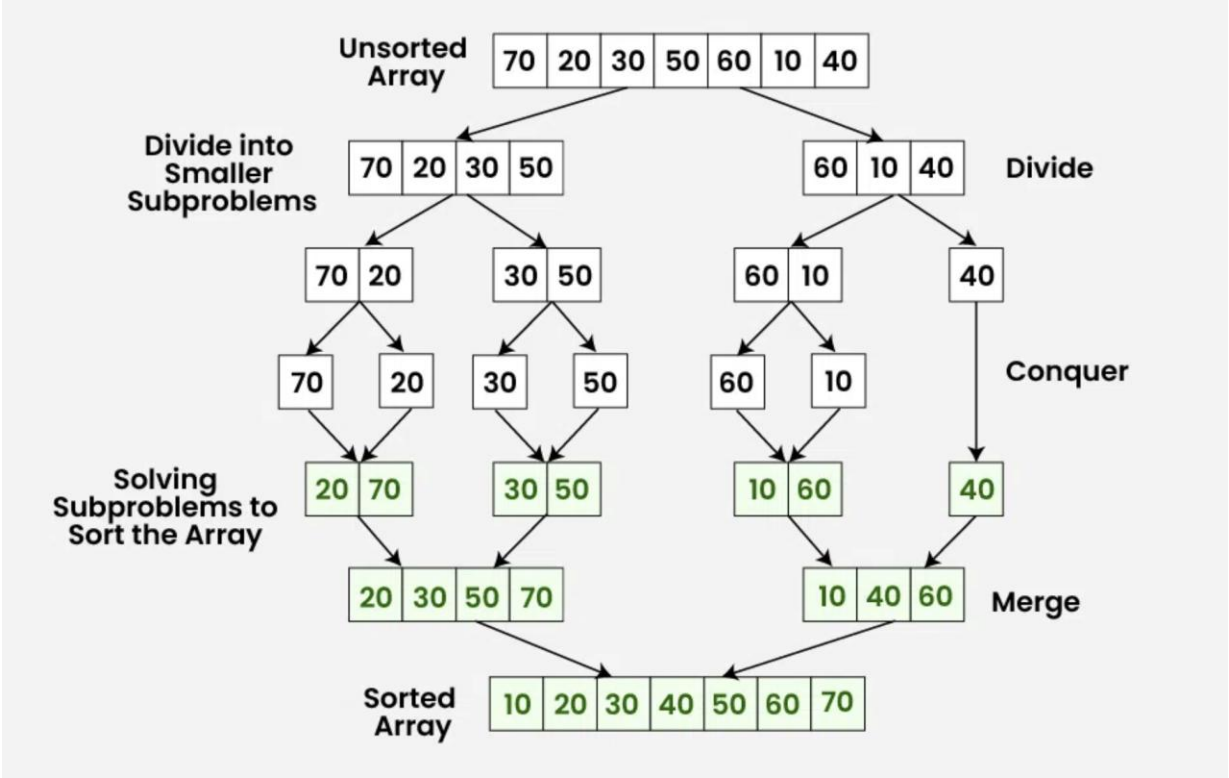
Combinar

Une las soluciones parciales para obtener la solución del problema original.

Eficiencia: mejora cuando combinar toma menos tiempo que resolver los subproblemas.

// divide y vencerás

Ejemplo: Merge Sort



`log2(n)` niveles de división × `O(n)` por nivel de merge = `O(n log n)`

// divide y vencerás

Ejemplo: búsqueda binaria

Búsqueda binaria recursiva

```
int binary_search(item_type s[], item_type key,
                  int low, int high) {
    if (low > high) return -1;
    int middle = (low + high) / 2;
    if (s[middle] == key) return middle;
    if (s[middle] > key)
        return binary_search(s, key, low, middle - 1);
    else
        return binary_search(s, key, middle + 1, high);
}
```

En cada paso

El problema se reduce a la mitad y no hay resultados que mezclar.

Complejidad

Un paso por cada mitad descartada.

O(log n)

// divide y vencerás

Encontrar el máximo de un arreglo

Solución trivial · O(n)

```
int findMax(int[] a, int n) {
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }
    return max;
}
```

Divide y vencerás · O(n)

```
static int findMax(int[] a, int lo, int hi) {
    if (lo > hi) return Integer.MIN_VALUE;
    if (lo == hi) return a[lo];
    int mid = (lo + hi) / 2;
    int leftMax = findMax(a, lo, mid);
    int rightMax = findMax(a, mid + 1, hi);
    return Math.max(leftMax, rightMax);
}
```

Investigar: ¿por qué la versión recursiva es `O(n)` y no `O(log n)`?

En la práctica: ambas son `O(n)`; la recursiva no es más rápida (llamadas y pila).

Paralelismo en divide y vencerás

1

Computación paralela

Realiza varias tareas a la vez, aprovechando múltiples núcleos de procesamiento.

2

En divide y vencerás

Cada partición del problema se asigna a un núcleo, lo que reduce el tiempo de ejecución.

Investigar: ¿cómo se puede implementar paralelismo en la búsqueda binaria y en Merge Sort?

Programación dinámica (DP)

«Programar» significa planificar: lo acuñó Richard Bellman en los años 50. No se refiere a escribir código.

Divide

Parte el problema en subproblemas, como divide y vencerás.

Memoiza

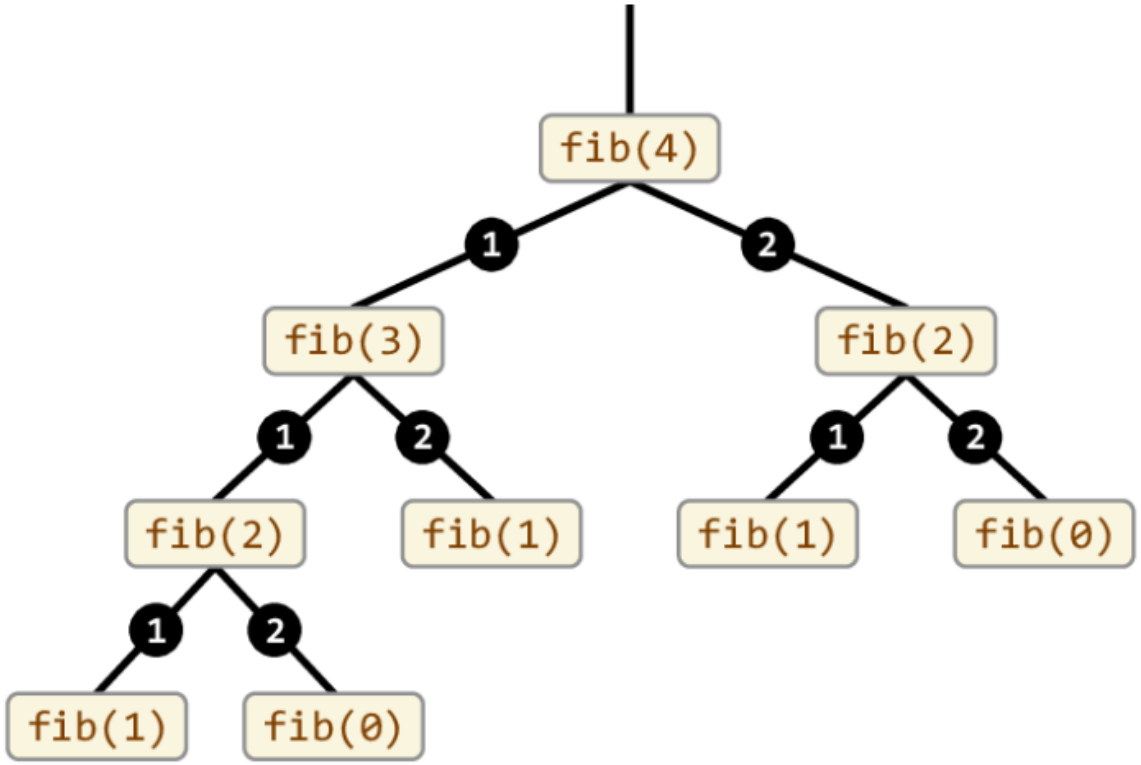
Recuerda la solución de cada subproblema para resolverlo una sola vez.

Se translapan

El resultado de un subproblema puede ayudar a resolver otros.

Memoización: optimizar recordando resultados. **Memorización:** poner algo en la memoria.

Fibonacci con el enfoque tradicional



Se calcula `fib(2)` dos veces y `fib(1)` tres veces: un claro desperdicio de recursos.

// programación dinámica

Fibonacci con programación dinámica

Se guarda cada resultado en **mem**

```
int fib(n) {
    mem[n + 2];  // por si n es 0 o 1
    mem[0] = 0;
    mem[1] = 1;
    for (i = 2; i < n + 1; i++) {
        mem[i] = mem[i - 1] + mem[i - 2];
    }
    return mem[n];
}
```

El arreglo **mem** para n = 4

i	0	1	2	3	4
mem	0	1	1	2	3

Divide y vencerás

Recalcula los mismos subproblemas.

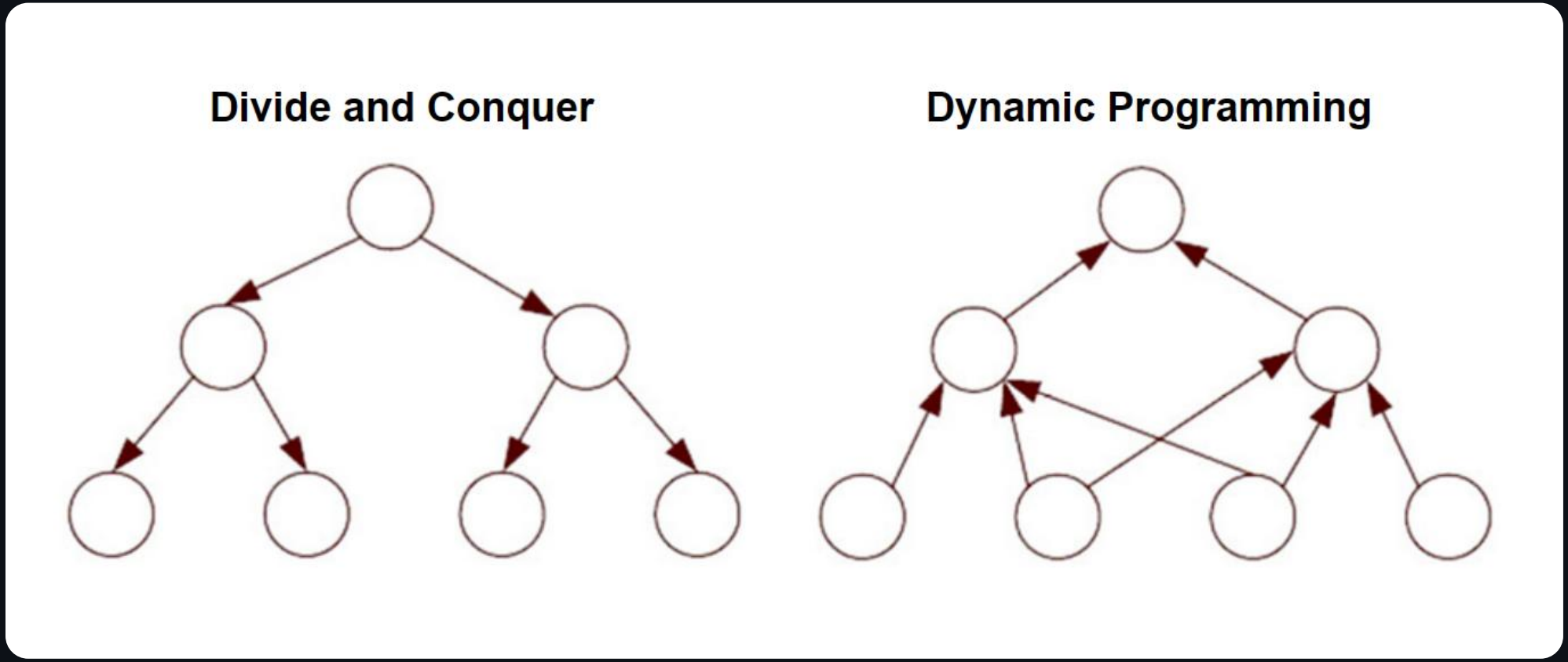
O(2^n)

Con DP

Cada subproblema se resuelve una vez.

O(n)

Divide y vencerás vs. programación dinámica



En D&V los subproblemas son independientes; en DP se comparten y se reutilizan.

¿Cuándo sirve la programación dinámica?

Subestructura óptima

La solución óptima se arma con soluciones óptimas de subproblemas.

Subproblemas repetidos

Pocos subproblemas distintos que se translapan: se resuelven una y otra vez.

Memoización (top-down)

Recursión con una tabla: antes de calcular, se consulta si ya está guardado.

Tabulación (bottom-up)

Se llena la tabla de menor a mayor, sin recursión, como el arreglo `mem`.

Contraste: en Merge Sort los subproblemas no se repiten; por eso le basta divide y vencerás.

// programación dinámica

LCS: subsecuencia común más larga

Enfoque tradicional (recursivo)

```
int lcs(String X, String Y, int m, int n) {
    if (m == 0 || n == 0)
        return 0;
    if (X.charAt(m - 1) == Y.charAt(n - 1))
        return 1 + lcs(X, Y, m - 1, n - 1);
    else
        return max(lcs(X, Y, m, n - 1),
                   lcs(X, Y, m - 1, n));
}
```

Dos cadenas de ejemplo

```
S1 = "BCDAACD"
S2 = "ACDBAC"
```

No es contigua

Puede saltar letras, a diferencia de una subcadena: BC, CDAC, DAC…

Sin memoización

Repite subproblemas: `O(2^(n+m))`.

LCS recursivo: subproblemas repetidos



Subproblema repetido

Se muestran solo los primeros niveles.

`(BCD, ACD)` se resuelve dos veces, y cada nivel duplica el trabajo: `O(2^(n+m))`.

LCS con programación dinámica

Se llena una matriz (m+1) × (n+1)

```
static int lcs(String X, String Y, int m, int n) {
  int[,] L = new int[m + 1, n + 1];
  for (int i = 0; i <= m; i++) {
    for (int j = 0; j <= n; j++) {
      if (i == 0 || j == 0)
        L[i, j] = 0;
      else if (X[i - 1] == Y[j - 1])
        L[i, j] = L[i - 1, j - 1] + 1;
      else
        L[i, j] = max(L[i - 1, j], L[i, j - 1]);
    }
  }
  return L[m, n];
}
```

L para **BCDA** (filas) y **ACDB** (columnas)

		A	C	D	B
	0	0	0	0	0
B	0	0	0	0	**1**
C	0	0	**1**	1	1
D	0	0	1	**2**	2
A	0	**1**	1	2	**2**

Las celdas en teal son letras iguales. Resultado: `L[m, n] = 2` (p. ej. CD) en tiempo `O(nm)`.

// programación dinámica

Mochila 0/1: ¿qué comer con 800 kcal?

Productos

	kcal	saciedad
A	100	2
B	200	5
C	400	6
D	500	10
E	700	13
F	800	16

Saciedad máxima según las kcal

Mejora la fila anterior · Óptimo

Con	100	200	300	400	500	600	700	800
A	2	2	2	2	2	2	2	2
A–B	2	**5**	**7**	**7**	**7**	**7**	**7**	**7**
A–C	2	5	7	7	**8**	**11**	**13**	**13**
A–D	2	5	7	7	**10**	**12**	**15**	**17**
A–E	2	5	7	7	10	12	15	17
A–F	2	5	7	7	10	12	15	**17**

Cada celda: el mayor entre copiar la de arriba y tomar el producto más la celda de arriba con las kcal restantes.

Óptimo: 17 (A + B + D). ¿Por qué las filas E y F no cambian nada?

// backtracking

Backtracking: probar y retroceder

Estructura general

```
backtrack(x) {
    if (x == solucion) {
        return true;
    }
    for (nodo in x.nodos) {
        if (backtrack(nodo)) {
            return true;
        }
    }
    return false;  // retroceder
}
```

Idea

Probar secuencias de decisiones hasta que una funcione.

Popularizado por Derrick H. Lehmer

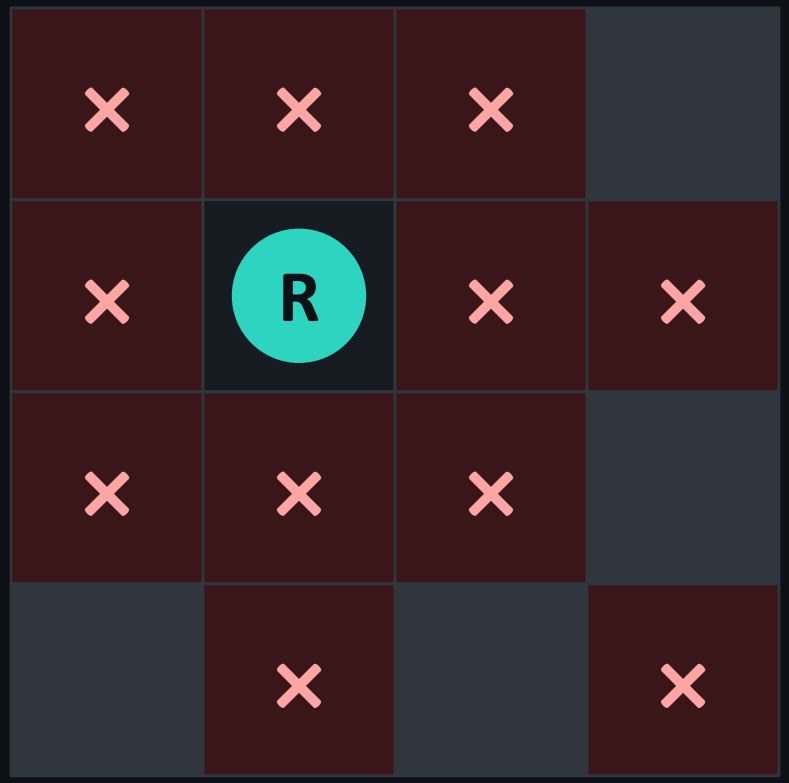
Árbol de decisiones

Cada nodo solo ve a sus hijos. Si uno lleva a error, se regresa y se prueba otro.

Si ningún hijo lleva a la solución, `return false` hace que el nodo anterior pruebe con otro hijo.

El problema de las N-reinas

Tablero de 4 × 4



Objetivo

Colocar N reinas en un tablero de N × N sin que se ataquen entre sí.

Una reina ataca…

Cualquier pieza en su misma fila, columna o diagonal.

En la figura

R es la reina; las casillas con × son las que ataca.

// backtracking

N-reinas con backtracking

Se coloca una reina por columna

```
boolean solve(int[][] board, int col) {
    if (col == N) {
        return true;
    }
    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;
            if (solve(board, col + 1)) {
                return true;
            }
            board[i][col] = 0;
        }
    }
    return false;
}
```

Avanzar

Si la fila `i` es segura, se coloca la reina y se resuelve la columna siguiente.

Retroceder

Si la siguiente falla, se quita la reina (`board[i][col] = 0`) y se prueba otra fila.

// backtracking

isSafe: ¿es segura esta casilla?

Revisa fila y diagonales, hacia la izquierda

```
boolean isSafe(int[][] board, int row, int col) {
    for (int j = 0; j < col; j++)
        if (board[row][j] == 1) return false;
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j] == 1) return false;
    for (int i = row, j = col; i < N && j >= 0; i++, j--)
        if (board[i][j] == 1) return false;
    return true;
}
```

Misma fila

¿Hay una reina a la izquierda en la fila `row`?

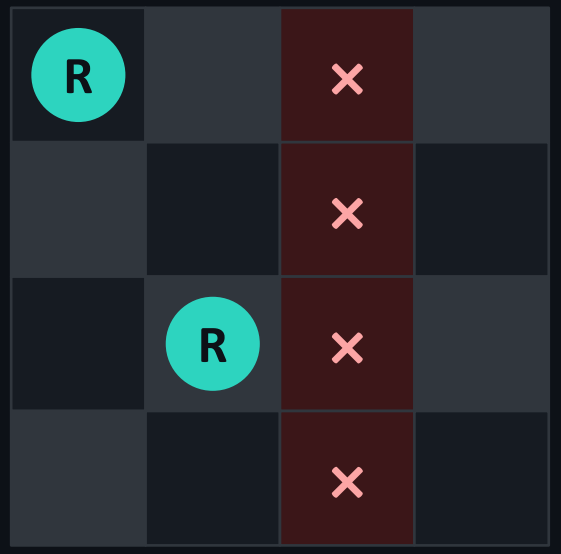
Dos diagonales

Una sube y otra baja, una fila por cada columna que retrocede.

Solo se mira a la izquierda: la columna actual y las siguientes todavía no tienen reinas.

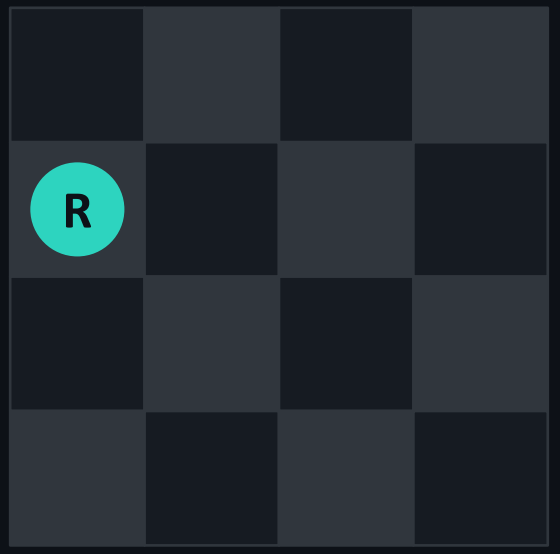
4 reinas: avanzar, retroceder, resolver

1 · Callejón sin salida



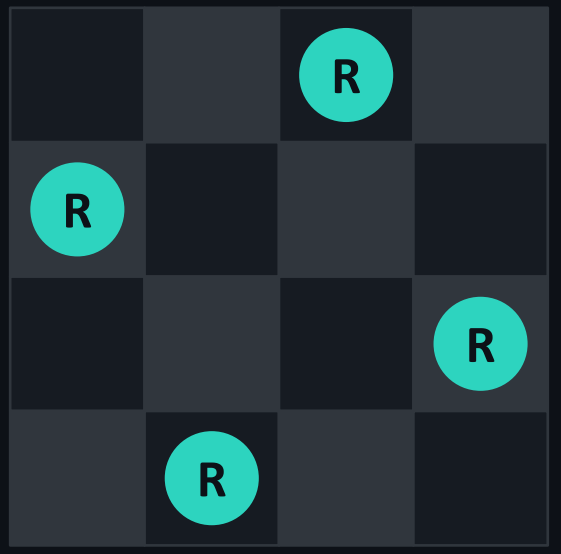
Ninguna fila es segura en la columna 2

2 · Retroceder



Se agotan las opciones y la primera reina cambia de fila

3 · Solución



Se llega a `col == N`

> Retroceder descarta toda una rama del árbol de decisiones sin explorarla completa.

Algoritmos ávidos (greedy)

Construyen la solución parte por parte, eligiendo siempre la pieza con el beneficio más obvio e inmediato.

Simples

Son sencillos de implementar.

Eficientes

Devuelven el resultado en poco tiempo.

No reconsideran

Nunca vuelven atrás sobre las opciones ya elegidas.

Ojo: no siempre dan la solución óptima, aunque en muchos casos es suficiente.

Ejemplo: cambio de 39 colones

Cambio restante	Moneda elegida	Nuevo restante
39	10	29
29	10	19
19	10	9
9	5	4
4	2	2
2	2	0

Regla: elegir la moneda más grande que no supere el cambio restante, restarla y repetir.

Resultado: 10 + 10 + 10 + 5 + 2 + 2 = **6 monedas**.

El enfoque ávido no siempre es óptimo

Ávido: 5 monedas

Cambio de 14 con monedas de 1, 7 y 10.
Toma siempre la mayor:

10 1 1 1 1

vs

Óptimo: 2 monedas

Dos monedas de 7 suman exactamente 14:

7 7

Cuidado: «lo mejor ahora» no siempre es «lo mejor al final».

Cuando el ávido falla: monedas con DP

Mínimo de monedas para cada monto

Mejora la fila anterior | Resultado

Monedas	0	1	2	3	4	5	6	7	8
{1}	0	1	2	3	4	5	6	7	8
{1, 4}	0	1	2	3	**1**	**2**	**3**	**4**	**2**
{1, 4, 6}	0	1	2	3	1	2	**1**	**2**	**2**

Ávido: 3 monedas

6 + 1 + 1

DP: 2 monedas

4 + 4

Costo

`O(monedas × monto)`

Cada celda: el menor entre la celda de arriba y 1 + la celda de su fila a «moneda» columnas a la izquierda.

Algoritmos probabilísticos

Usan aleatoriedad para mejorar el desempeño, sacrificando la confiabilidad. Pueden:

1

No dar resultado

No se produce ningún resultado.

2

Resultado incorrecto

Se produce una respuesta equivocada.

3

Respuesta aproximada

Se produce un resultado aproximado.

> Ejecuciones distintas pueden producir respuestas distintas.

Monte Carlo vs. Las Vegas

Monte Carlo

- **Siempre** retorna un resultado
- Puede **no ser correcto**
- Se intenta minimizar la probabilidad de error
- Más ejecuciones reducen esa probabilidad

vs

Las Vegas

- **Siempre** retorna un resultado correcto
- Puede **no producir resultado**
- Sin cota superior de tiempo de ejecución
- Más ejecuciones reducen la probabilidad de no obtener resultado

> Monte Carlo arriesga la **corrección**; Las Vegas arriesga **no obtener resultado**.

// probabilísticos

Aleatoriedad en el computador

Pseudo-aleatorio

- Parece aleatorio, pero es **determinista**
- Parte de un valor inicial: la **semilla** (seed)
- Cada número se genera a partir del anterior
- Puede haber ciclos: un número se repite

vs

Cuadrado medio

Eleve la semilla (d dígitos) al cuadrado, complete a 2d dígitos y tome los d del medio.

```
1234^2 = 01522756 -> 5227
5227^2 = 27321529 -> 3215
3215^2 = 10336225 -> 3362
```

> Una fuente de aleatoriedad física es la desintegración radiactiva (otra: el ruido térmico).

Monte Carlo: elemento mayoritario

Se prueba un elemento elegido al azar

```
boolean tieneElementoMayoritario(array, n) {
    i = random(0, n - 1);
    x = array[i];
    k = 0;
    for (j = 0; j < n; j++) {
        if (array[j] == x) {
            k++;
        }
    }
    return k > n / 2;
}
```

El problema

¿Algún elemento aparece más de `N/2` veces? La solución trivial es `O(N^2)`.

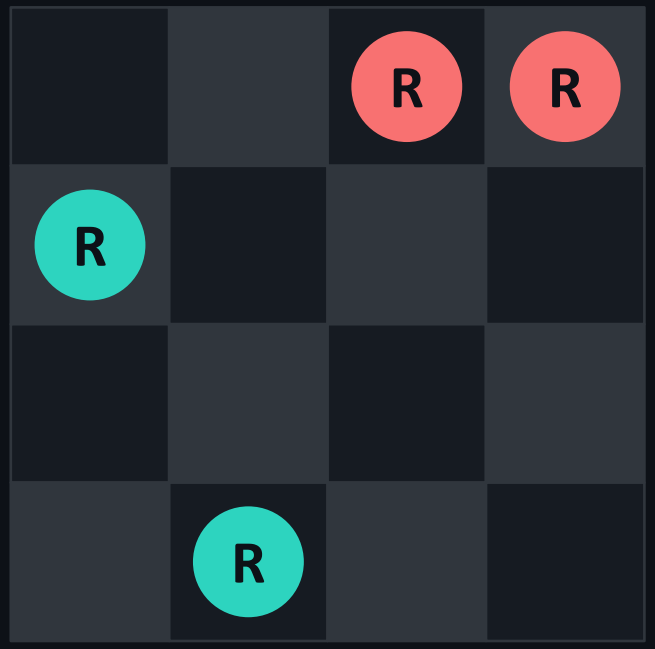
Probabilidad de error

Una ejecución: menor a `1/2`. Después de `k` ejecuciones: `1/2^k`.

Puede devolver `false` aunque exista un mayoritario, si el índice al azar no lo contiene.

Las Vegas: N-reinas al azar

Un intento al azar



1. Colocar una reina al azar en cada columna.
2. Verificar si hay colisiones.
3. Si no hay colisiones: retornar la solución.
4. Si las hay: repetir el proceso.

Siempre es correcto, pero sin cota de tiempo; si hay muchas soluciones, se encuentra una pronto.

// probabilísticos

Estructuras de datos probabilísticas

Aproximadas

Responden consultas sobre grandes conjuntos de datos con respuestas aproximadas.

Eficiencia

Sacrifican precisión para ganar eficiencia temporal.

Hashing y azar

Los usan para reducir la complejidad espacial y temporal.

Diferencia clave: los algoritmos probabilísticos pueden devolver resultados correctos; estas estructuras no dan respuestas definitivas.

// probabilísticos

Filtros de Bloom: ¿existe el username?

Soluciones simples

- Búsqueda secuencial: `O(N)`, muy lenta con millones de usuarios
- Búsqueda binaria: mejor, pero exige usuarios ordenados y en memoria

vs

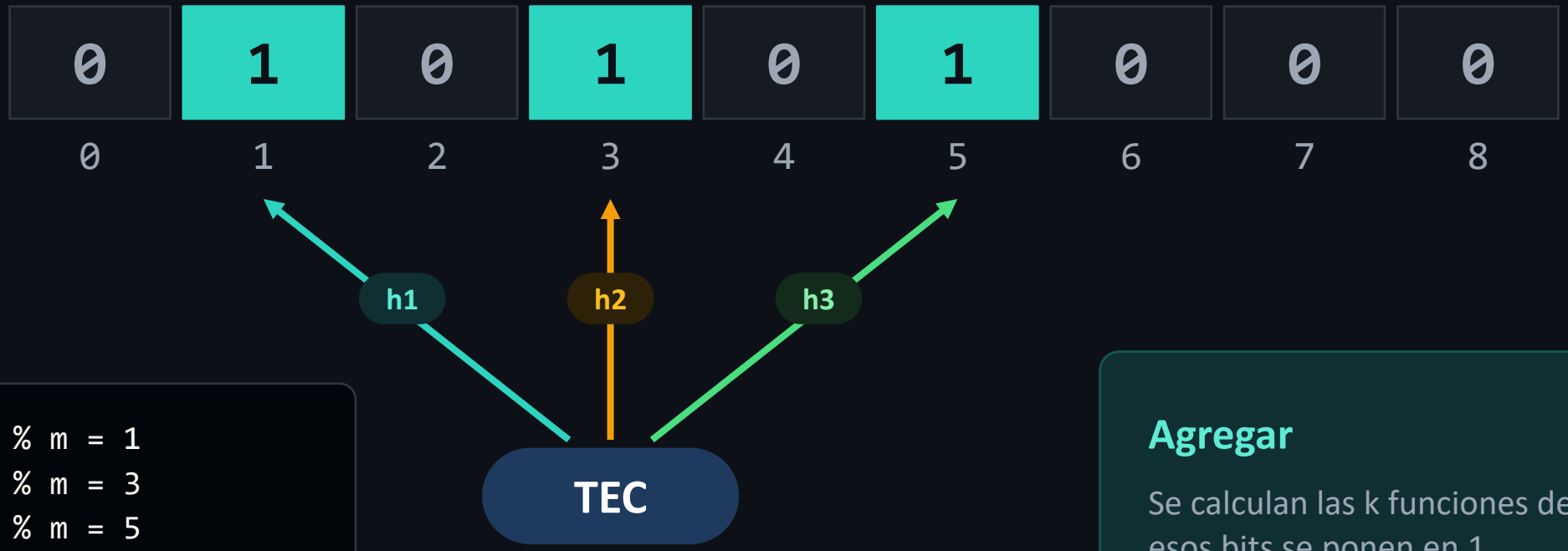
Filtro de Bloom

- Un array de bits de tamaño fijo representa todo el conjunto
- Hay falsos positivos, **nunca** falsos negativos
- Agregar nunca falla; al llenarse, aumentan los falsos positivos
- No se pueden eliminar elementos

> «No existe» siempre es cierto; «ya existe» puede ser un falso positivo.

Filtro de Bloom: agregar «TEC»

Un arreglo de `m = 9` bits (todos en 0) y `k = 3` funciones de hash.



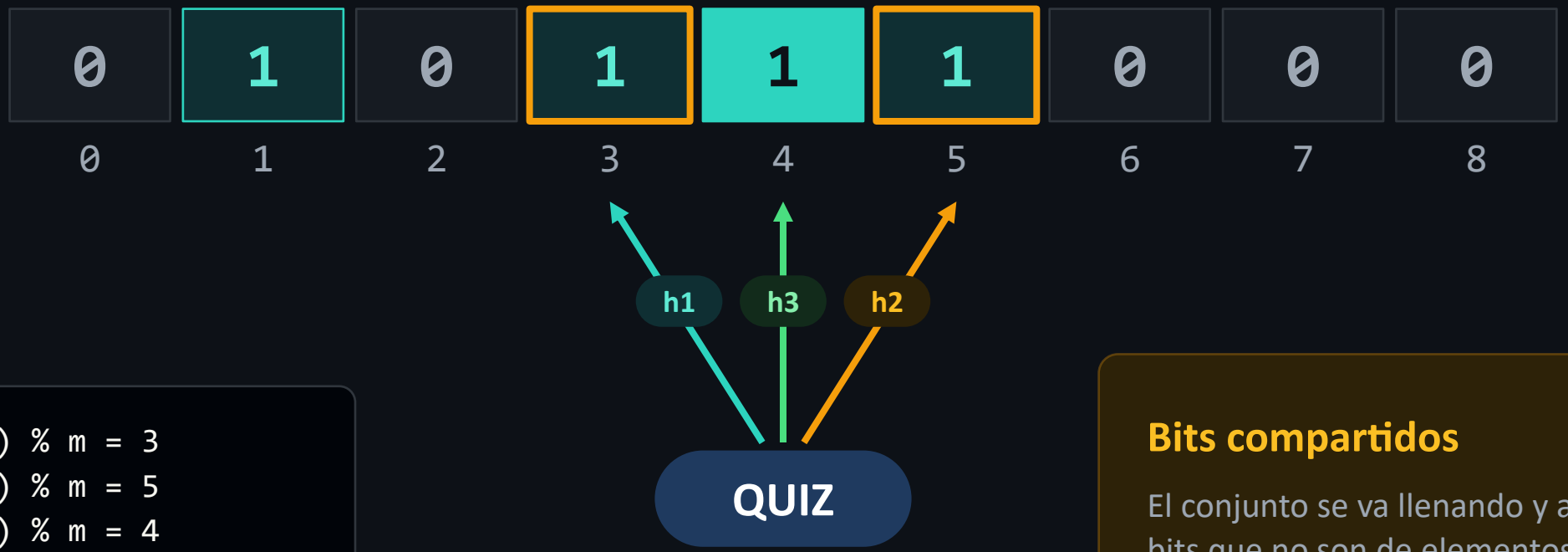

```
h1(TEC) % m = 1
h2(TEC) % m = 3
h3(TEC) % m = 5
```

Agregar

Se calculan las k funciones de hash y esos bits se ponen en 1.

Filtro de Bloom: agregar «QUIZ»

Los bits 3 y 5 ya estaban en 1: «QUIZ» solo activa el bit 4.



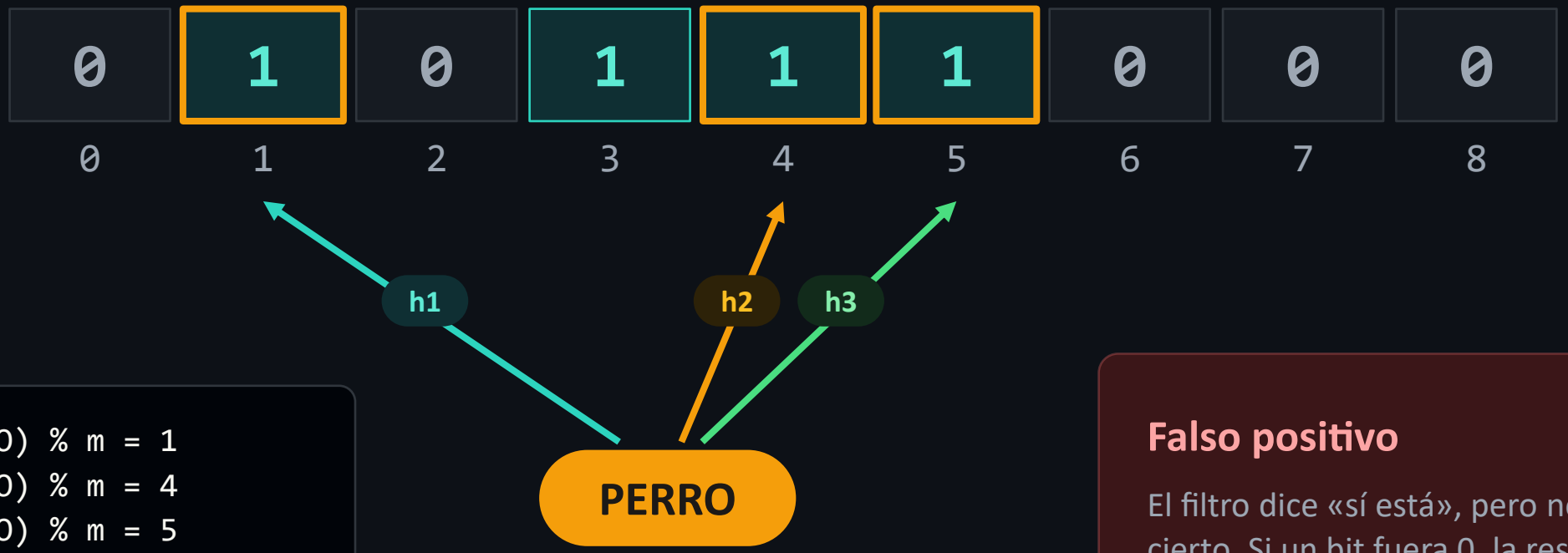

```
h1(QUIZ) % m = 3
h2(QUIZ) % m = 5
h3(QUIZ) % m = 4
```

Bits compartidos

El conjunto se va llenando y activa bits que no son de elementos distintos.

Filtro de Bloom: buscar «PERRO»

Los tres bits consultados ya están en 1, aunque «PERRO» nunca se agregó.




```
h1(PERRO) % m = 1
h2(PERRO) % m = 4
h3(PERRO) % m = 5
```

Falso positivo

El filtro dice «sí está», pero no es cierto. Si un bit fuera 0, la respuesta sería «no está».

Filtro de Bloom: complejidad

Inserción

Calcular k hashes.

O(k)

Búsqueda

Calcular k hashes.

O(k)

Espacio

Un arreglo de m bits.

O(m)

Función hash

Deben ser rápidas; las criptográficas son estables pero lentas.

Falso positivo con n elementos: `(1 - (1 - 1/m)^(k·n))^k`

Aplicaciones de los filtros de Bloom

Medium.com

Identifica los posts que el usuario ya vio.

Cloudflare

Identifica direcciones IP maliciosas.

Google Chrome

Identifica URLs maliciosas.

Apache Cassandra

Busca registros inexistentes en disco.

¿Qué es un algoritmo genético?

Basados en el trabajo de John Holland (1962) y en la evolución de Darwin.

Búsqueda

Técnica para optimización y aprendizaje automático que simula la selección natural.

Heurístico

No garantiza el óptimo; suele dar buenas soluciones en un tiempo razonable, sin garantía formal.

Inspiración

Selección natural, reproducción y mutación.

Heurística: en griego significa «encontrar», la raíz de «eureka».

Definiciones esenciales

Individuo

Una solución al problema. Suele ser un bit-vector.

Población

Conjunto de individuos o soluciones posibles.

Fitness

Función que evalúa la calidad de un individuo.

Características

Propiedades del individuo; pueden ser sus genes.

Genoma

Conjunto de características. En un bit-vector: 1 = presente, 0 = ausente.

Cromosomas

Según el problema, un individuo puede usar varios cromosomas.

GA sobre la mochila: bit-vector y fitness

Productos

	kcal	saciedad
A	100	2
B	200	5
C	400	6
D	500	10
E	700	13
F	800	16

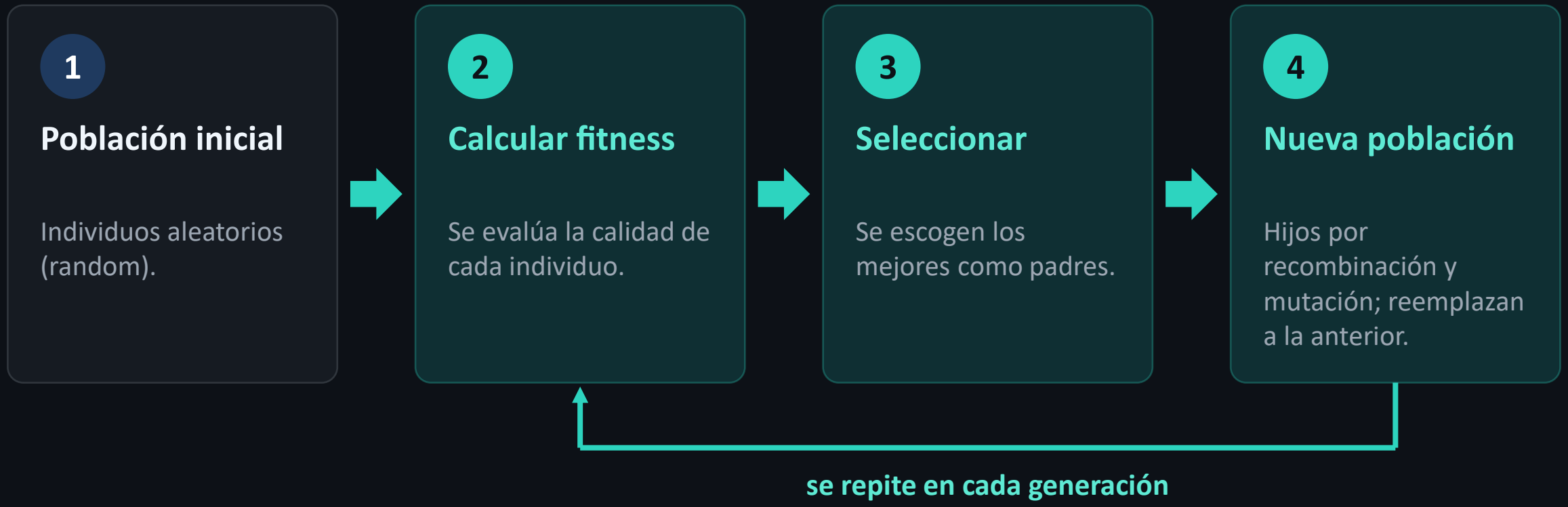
Cuatro individuos de la población

Lleva	A	B	C	D	E	F	kcal	fitness
A+B+D	1	1	0	1	0	0	800	17
F	0	0	0	0	0	1	800	16
A+B	1	1	0	0	0	0	300	7
A+C+D	1	0	1	1	0	0	1000	0

Fitness: la saciedad total si las kcal no pasan de 800; si pasan, 0 (individuo inválido).

Sin garantía: la tabla de DP asegura el 17; el GA solo busca valores cercanos.

Funcionamiento de un algoritmo genético



¿Cuándo se detiene? Al llegar a un máximo de generaciones, a un fitness mínimo o máximo, o cuando la población ya no cambia. Una población inicial muy pequeña puede quedar en un óptimo local.

Recombinación y mutación

	0	1	2	3	4	5
P1	0	0	0	0	0	0
P2	1	1	1	1	1	1
H1	0	0	0	1	1	1
H2	1	1	1	0	0	0
H1 mutado	0	1	0	1	1	1

recombinación

mutación (flip)

Punto de cruce (cross-over)

Recombinar mezcla los genes de los padres; mutar invierte bits al azar, según una probabilidad.

// cierre

Resumen

- **Divide y vencerás:** dividir, conquistar y combinar (Merge Sort, búsqueda binaria).
- **Programación dinámica:** reutilizar subproblemas que se translapan (LCS, mochila, monedas).
- **Backtracking:** probar decisiones y retroceder si fallan (N-reinas).
- **Algoritmos ávidos:** elegir lo mejor en cada paso, sin reconsiderar (monedas).
- **Probabilísticos:** Monte Carlo, Las Vegas y filtros de Bloom.
- **Genéticos:** población, fitness, selección, recombinación y mutación (mochila).

// cierre

Para investigar y referencias

- ¿Por qué `findMax` por divide y vencerás es `O(n)` y no `O(log n)`?
  Pista: cuente cuántas llamadas recursivas se generan para n = 4 y para n = 8.
- ¿Cómo se puede implementar paralelismo en la búsqueda binaria y en Merge Sort?
  Pista: ¿qué partes del problema son independientes entre sí?
- Resuelva la mochila (800 kcal) con backtracking.
  Pista: en cada nivel del árbol, ¿lleva o no el producto? ¿Cuándo retrocede?
- ¿Qué paradigmas sirven para la mochila, las N-reinas y las monedas?
  Pista: un mismo problema admite varios paradigmas; compare su corrección, tiempo y simplicidad.

Referencias: GeeksforGeeks. «Introduction to Divide and Conquer Algorithm» (geeksforgeeks.org/introduction-to-divide-and-conquer-algorithm). GeeksforGeeks. «Introduction to Greedy Algorithm: Data Structures and Algorithm Tutorials» (geeksforgeeks.org/introduction-to-greedy-algorithm-data-structures-and-algorithm-tutorials).